

Proyecto 2

Indicaciones:

- *Los estudiantes deberán trabajar en grupos y desarrollar uno de los proyectos propuestos en el curso.*
- *Cada proyecto deberá ser implementado de manera funcional, evidenciando la correcta aplicación de los conceptos de compiladores y lenguajes de programación desarrollados durante el curso.*
- *El desarrollo deberá incluir documentación técnica clara y estructurada, describiendo la arquitectura del sistema, las decisiones de diseño, las tecnologías utilizadas y los principales componentes implementados.*
- *Cada grupo deberá realizar una exposición técnica del proyecto, demostrando el funcionamiento del sistema y explicando adecuadamente los componentes desarrollados, las decisiones de implementación y los resultados obtenidos.*
- *Se recomienda el uso de herramientas modernas y buenas prácticas de ingeniería de software para la organización, implementación y documentación del proyecto.*
- *Todos los integrantes del equipo deberán participar activamente en el desarrollo del proyecto, demostrando contribuciones reales, verificables y consistentes con el trabajo presentado.*
- *Todos los integrantes del grupo deberán demostrar capacidad de análisis, investigación, implementación y comprensión profunda de los conceptos de compiladores y lenguajes de programación involucrados en el proyecto.*
- *El uso de herramientas de inteligencia artificial estará permitido únicamente como apoyo complementario durante el desarrollo del proyecto. Sin embargo, el trabajo deberá reflejar comprensión técnica real por parte de todos los miembros del grupo. En caso de evidenciarse dependencia excesiva de herramientas de IA, incapacidad para explicar el funcionamiento del código presentado o desconocimiento de los componentes implementados por parte de uno o más integrantes, el proyecto será calificado con cero (0).*
- *Los grupos deberán cumplir con los objetivos mínimos establecidos en el desarrollo del proyecto. En caso de no alcanzarlos, el proyecto será calificado con cero (0).*
- *En caso de detectarse que uno o más integrantes no han participado de manera efectiva en el desarrollo del trabajo, el proyecto será calificado con cero (0).*

Estudio de Runtimes y Backends de Compilación Modernos con Wasmtime y Cranelift

El presente proyecto tiene como objetivo comprender los fundamentos de los runtimes y backends de compilación modernos mediante el estudio del ecosistema de *Wasmtime* y *Cranelift*, explorando tecnologías actuales utilizadas en sistemas de ejecución basados en WebAssembly y en procesos modernos de generación de código. A lo largo del desarrollo del proyecto, los estudiantes analizarán la arquitectura y funcionamiento interno de herramientas utilizadas en entornos reales de compilación, familiarizándose además con dinámicas y metodologías propias de comunidades *open source* vinculadas al desarrollo de compiladores, runtimes y máquinas virtuales modernas. Los estudiantes deberán:

- Comprender la arquitectura general de WebAssembly, *Wasmtime* y *Cranelift*.
- Analizar el funcionamiento de runtimes modernos y sistemas de generación de código JIT/AOT.
- Estudiar el flujo completo de compilación y ejecución de programas basados en WebAssembly.
- Explorar repositorios, documentación técnica, *issues* y herramientas del ecosistema *open source*.
- Familiarizarse con prácticas de desarrollo colaborativo utilizadas en proyectos modernos de compiladores y runtimes.
- Investigar componentes internos relacionados con representación intermedia, optimización y generación de código.
- Desarrollar capacidades de investigación, análisis técnico y lectura de código fuente de gran escala.
- Elaborar un reporte técnico que documente los conceptos estudiados, experimentos realizados y hallazgos obtenidos.
- Comunicar de manera técnica y estructurada los resultados y aprendizajes del proyecto mediante una exposición final.

El máximo logro del proyecto consistirá en que el grupo realice una contribución significativa dentro del ecosistema de *Wasmtime*, *Cranelift* o proyectos relacionados, que sea reconocida, aceptada o incorporada por la comunidad open source correspondiente. En este sentido, aquellos grupos que logren una contribución validada podrán obtener hasta 3 puntos adicionales en el Examen final del curso. Se considerarán como aportes válidos:

- Pull Requests aceptados: Cambios incorporados oficialmente al proyecto por los mantenedores. Incluye corrección de código, mejoras pequeñas, refactorización, documentación integrada, nuevos ejemplos y mejoras de tooling.
- Corrección de bugs: Identificación y solución de errores existentes en el proyecto. Incluye errores de compilación, fallos en ejecución, problemas de compatibilidad, errores en tests y comportamiento incorrecto del runtime o backend.

- Mejoras de documentación: Contribuciones orientadas a mejorar la comprensión del proyecto. Incluye tutoriales, corrección de documentación, ejemplos explicativos, guías de instalación, diagramas de arquitectura y aclaración de APIs.
- Tooling: Desarrollo de herramientas auxiliares para el ecosistema. Incluye scripts, herramientas CLI, analizadores, visualizadores, automatización de pruebas, debugging tools e integración con editores.
- Mejoras técnicas: Optimización o mejora de componentes existentes. Incluye mejoras de rendimiento, simplificación de código, optimización interna y reducción de consumo de recursos.
- Issues relevantes reconocidos por mantenedores: Reportes o análisis técnicos útiles para la comunidad. Incluye detección de errores reproducibles, análisis técnico, propuestas de mejora y reportes bien documentados.
- Participación técnica significativa en discusiones: Intervenciones técnicas relevantes dentro de la comunidad. Incluye discusión de arquitectura, revisión de propuestas, ayuda en resolución de errores y aportes técnicos en foros oficiales.

La rúbrica de calificación del proyecto se basa en los siguientes criterios:

Criterio	Excelente	Bueno	Regular	Deficiente	Pts
Comprensión Wasm	Explica claramente la arquitectura y relación entre runtime y WebAssembly.	Comprende adecuadamente los conceptos con pequeñas imprecisiones.	Presenta comprensión parcial o superficial.	No demuestra comprensión técnica.	5
Comunidad Open Source	Participación activa mediante revisión de repositorios y discusiones.	Explora adecuadamente repositorios y documentación principal.	Exploración limitada de la comunidad y sus herramientas.	No evidencia interacción con la comunidad.	6
Análisis Arquitectura	Analiza correctamente componentes y flujo de compilación.	Presenta un análisis adecuado aunque limitado en profundidad.	El análisis es superficial o incompleto.	No presenta análisis técnico relevante.	4
Reporte Técnico	Reporte bien estructurado, claro y con buenas referencias.	Reporte organizado con algunos problemas menores.	Reporte limitado, poco claro o incompleto.	Reporte desordenado o insuficiente.	2
Exposición	Expone con claridad, dominio conceptual y buena organización.	Presenta adecuadamente con algunas dificultades menores.	Presentación poco clara o con vacíos.	No logra comunicar el trabajo.	3

Implementación de Lenguajes Extensibles mediante Adaptable Parsing Expression Grammars en Haskell

El presente proyecto tiene como objetivo comprender los fundamentos de las Parsing Expression Grammars (PEG) y de las Adaptable Parsing Expression Grammars (APEG), analizando su uso como base formal para el diseño de lenguajes extensibles y sistemas de parsing modernos. A lo largo del desarrollo del proyecto, los estudiantes estudiarán mecanismos de extensión dinámica de gramáticas, así como el modelo formal propuesto en la literatura especializada, con énfasis en su implementación práctica. Se abordará además la construcción de parsers basados en combinadores, así como el uso de estructuras funcionales avanzadas propias del lenguaje *Haskell*. Los estudiantes deberán:

- Comprender los fundamentos de las Parsing Expression Grammars (PEG) y las Adaptable Parsing Expression Grammars (APEG).
- Analizar mecanismos de extensión dinámica de gramáticas y lenguajes de programación.
- Estudiar el modelo formal y la implementación presentada en la literatura especializada (paper base del curso).
- Comprender el uso de *parsing combinators* y *State Monads* en *Haskell*.
- Analizar la construcción dinámica de árboles de sintaxis abstracta (AST) y reglas gramaticales.
- Implementar componentes de un lenguaje extensible utilizando *Haskell*.
- Explorar técnicas modernas de metaprogramación y *language-oriented programming*.
- Desarrollar capacidades de investigación, lectura de papers e implementación de sistemas de compilación modernos.
- Elaborar un reporte técnico que documente el sistema desarrollado y el análisis realizado.
- Comunicar de manera técnica y estructurada los resultados y aprendizajes obtenidos mediante una exposición final.

El máximo logro del proyecto consistirá en el desarrollo de una extensión para *Visual Studio Code* orientada a lenguajes extensibles, incorporando funcionalidades avanzadas de análisis, asistencia de código y visualización del lenguaje. En este sentido, los grupos que logren una implementación podrán obtener hasta 3 puntos adicionales en el Examen final del curso. Se considerarán extensiones válidas:

- Syntax Highlighting: Implementación de resaltado sintáctico para un DSL, incluyendo coloreado de keywords, operadores, comentarios, tipos y bloques.
- Parser integrado: Desarrollo de un parser funcional dentro de la extensión, con detección de errores sintácticos, validación de estructuras y análisis de tokens.
- Autocompletado: Implementación de sugerencias automáticas en el editor, incluyendo keywords, funciones, variables, tipos y snippets inteligentes.
- AST Viewer: Visualización de árboles sintácticos generados por el parser, de forma gráfica o estructurada.

- Semantic Analysis: Implementación de validaciones semánticas, como variables no declaradas, incompatibilidad de tipos, redefiniciones o validación de unidades.

La rúbrica de calificación del proyecto se basa en los siguientes criterios:

Criterio	Excelente	Bueno	Regular	Deficiente	Pts
Comprensión PEG/APEG	Explica claramente los fundamentos de PEG, APEG y extensibilidad de gramáticas.	Comprende adecuadamente los conceptos principales con pequeñas imprecisiones.	Presenta comprensión parcial o superficial de los conceptos.	No demuestra comprensión suficiente de PEG y APEG.	5
Análisis del Paper	Analiza correctamente el modelo formal, parsing combinators, monads y arquitectura propuesta.	Presenta un análisis adecuado aunque limitado en profundidad.	El análisis es superficial o incompleto.	No presenta análisis técnico relevante del paper.	5
Implementación Técnica	Implementa correctamente componentes funcionales del parser, DSL o mecanismos de extensibilidad.	Implementación funcional con algunos problemas menores.	Implementación incompleta o limitada.	No logra implementar componentes funcionales.	5
Reporte Técnico	Reporte bien estructurado, técnico, claro y con análisis profundo.	Reporte organizado con algunos problemas menores de claridad o profundidad.	Reporte limitado, poco claro o incompleto.	Reporte desordenado o insuficiente.	2
Exposición	Expone con claridad, dominio conceptual y buena organización técnica.	Presenta adecuadamente el tema con algunas dificultades menores.	Presentación poco clara o con vacíos conceptuales.	No logra comunicar adecuadamente el trabajo realizado.	3

Diseño e Implementación de un Compilador x86 con Optimización y Evaluación Comparativa

El objetivo del presente proyecto es diseñar e implementar un compilador completo para un lenguaje de programación sobre arquitectura x86, integrando las etapas fundamentales del proceso de compilación, desde el análisis léxico y sintáctico hasta la generación de código ensamblador y la optimización básica. Finalmente, el proyecto tiene como propósito fortalecer la capacidad de análisis, diseño e implementación de sistemas de software complejos, así como promover la evaluación experimental mediante la comparación del compilador desarrollado con herramientas de uso extendido en la industria. Los estudiantes deberán:

- Seleccionar un lenguaje de programación con características bien definidas y documentación formal. Este debe incluir, como mínimo, las siguientes características:
 - Básicas:
 - Tipos de datos básicos y definidos por el usuario.
 - Variables y manejo de alcance (*scope*).
 - Funciones.
 - Estructuras de control
 - **Struct**, arreglos y cadenas de caracteres (**strings**).
 - Avanzadas:
 - Punteros, direccionamiento de memoria y manejo de memoria dinámica.
 - Tipos genéricos y plantillas (*templates*).
 - Inferencia, conversión y promoción automática de tipos.
 - Arreglos multidimensionales y funciones **lambda**.
- Implementar un compilador completo que incluya las fases de análisis léxico, sintáctico y semántico.
- Implementar un sistema de verificación de tipos y manejo de errores léxicos, sintácticos y semánticos.
- Generar código ensamblador para arquitectura x86.
- Aplicar técnicas básicas de optimización sobre el código generado.
- Desarrollar un conjunto de *benchmarks* para evaluar el rendimiento del compilador.
- Realizar una comparación experimental con compiladores de uso extendido como GCC, Clang/LLVM, MSVC, Rustc o Go Compiler.
- Analizar y documentar los resultados obtenidos mediante tablas, gráficos y discusión técnica.
- Presentar y defender el proyecto mediante una exposición técnica.

El máximo logro del proyecto consistirá en la implementación de una aplicación funcional del compilador desarrollado que permita demostrar, de manera integrada, las distintas fases del proceso de compilación y las principales características del lenguaje implementado. Los grupos que logren desarrollar dicha aplicación podrán obtener hasta 3 puntos adicionales en la evaluación final del curso. La aplicación deberá incluir, como mínimo, los siguientes componentes:

- Editor de código para el lenguaje diseñado.
- Visualización del AST (*Abstract Syntax Tree*).
- Generación de código ensamblador x86.
- Ejecución o simulación del programa compilado.
- Visualización de resultados de ejecución.

La rúbrica de calificación del proyecto se basa en los siguientes criterios:

Criterio	Excelente	Bueno	Regular	Deficiente	Pts
Diseño y Lexer	Lenguaje consistente y lexer robusto con manejo adecuado de errores.	Diseño adecuado y lexer funcional con pocos errores.	Diseño limitado o lexer con fallos ocasionales.	Diseño incompleto o lexer incorrecto.	2
Sintaxis y AST	Parser correcto, AST bien estructurado y tabla de símbolos organizada.	Implementación funcional con leves limitaciones.	Implementación parcial o poco organizada.	Implementación incorrecta o incompleta.	2
Análisis Semántico	Verificación sólida de tipos y errores.	Buen manejo semántico.	Validación parcial.	Escasa validación.	1
Generación x86	Código eficiente, correcto y optimizado.	Código funcional con leves limitaciones.	Código poco eficiente.	Código incorrecto o incompleto.	3
Características Avanzadas	Implementa correctamente la mayoría de funcionalidades avanzadas.	Implementa varias funcionalidades relevantes.	Implementación parcial.	Muy pocas funcionalidades implementadas.	2
Optimización	Implementa optimizaciones relevantes y demuestra mejoras medibles.	Implementa algunas optimizaciones funcionales.	Optimización mínima.	No implementa optimizaciones.	3
Comparación Comercial	Benchmark riguroso y análisis sólido frente a compiladores comerciales.	Comparación adecuada con métricas parciales.	Comparación superficial.	No realiza comparación.	2
Reporte Técnico	Documentación técnica completa, clara y bien estructurada.	Documentación adecuada con algunos vacíos menores.	Documentación parcial o poco clara.	Documentación insuficiente.	2
Exposición	Presentación clara, dominio técnico y demostración sólida del compilador.	Buena presentación y explicación adecuada del funcionamiento.	Presentación limitada o con poco dominio del proyecto.	Escaso dominio o incapacidad de comunicar el trabajo.	3